

Projet IA — Principes et Protocole

Export automatique

2026-05-20

Contenu (markdown brut)

Le document suivant contient le contenu combiné des deux fichiers Markdown originaux.
Pour un rendu typographique avancé, utilisez pandoc pour convertir `combined_for_pdf.mdenPDF`.

```
% Projet IA - Principes et Protocole
% Auteur: Exports
% Date: 2026-05-20
```

```
\newpage
```

```
AA# Principes pour débutants - Agents IA et benchmark
```

Ce document explique, depuis les bases, les concepts clés du projet et des frameworks d'

```
## 1. Qu'est-ce qu'un "agent IA" ?
```

- Un agent IA est un programme qui reçoit une demande (prompt) et tente d'accomplir une
- Imaginez un assistant humain : vous lui posez une question, il cherche des information

```
## 2. Concepts de base (expliqués simplement)
```

- Modèle de langage (LLM) : c'est le "cerveau" qui génère du texte. Il prédit des mots p
- Prompt : la consigne que vous donnez au modèle (question + instructions). C'est comme
- Token : morceau de texte (mot ou partie de mot) utilisé pour mesurer la longueur d'une
- Température : règle qui contrôle la créativité du modèle. Plus elle est élevée, plus l
- Tool / Outil : une fonction externe que l'agent peut appeler (API, recherche web, calc

```
## 3. Pourquoi utiliser un framework d'agents ?
```

- Les frameworks fournissent des composants prêts à l'emploi : orchestration, gestion d'
- Ils aident à structurer des tâches compliquées (plusieurs étapes, plusieurs appels d'o

```
## 4. Brève présentation des frameworks (idée générale)
```

- LangChain : orienté chaînes (chains) et pipelines ; bon pour construire des séquences
- CrewAI : conçu pour des workflows avec rôles et responsabilités ; utile quand plusieurs
- AutoGen : puissant pour les scénarios multi-agents interactifs et conversationnels ; f
- LlamaIndex : spécialisé dans la récupération et la synthèse d'informations à partir d'

```
## 5. Types de tâches que l'on va benchmarker
```

- Recherche d'information : trouver une réponse précise.
- Raisonnement séquentiel : résoudre un problème en plusieurs étapes.
- Récupération documentaire : répondre à partir d'un corpus donné.
- Multi-agents : collaboration entre agents spécialisés.
- Gestion d'erreurs : inputs incomplets ou outils défectueux.

```
## 6. Comment fonctionne une exécution simple (pas-à-pas)
```

1. Vous posez la question (prompt).

2. L'agent analyse la demande et décide d'une stratégie (ex : chercher, calculer, synthèse).
3. Si nécessaire, il appelle un outil (ex : recherche web) et récupère des données.
4. Il intègre les résultats et produit une réponse finale.

Exemple concret très simple : "Quel est l'horaire d'ouverture de la bibliothèque municipale ?"

- Étape 1 : Agent lit la question.
- Étape 2 : Agent appelle l'outil de recherche web.
- Étape 3 : Récupère la page contenant l'horaire.
- Étape 4 : Synthétise et répond : "La bibliothèque est ouverte de 9h à 18h du mardi au dimanche."

7. Variables expérimentales (à maîtriser pour un benchmark fiable)

- Même modèle sous-jacent (même version du LLM).
- Même température et limites de tokens.
- Même jeux d'outils et mêmes timeouts.
- Même format d'entrée/sortie et même nombre d'essais.

Ces contrôles empêchent des biais liés à la configuration.

8. Métriques expliquées simplement

- Taux de réussite : proportion de tâches entièrement correctes.
- Complétion partielle : la tâche est partiellement résolue.
- Latence : temps pris pour répondre.
- Nombre d'appels outils : combien de fois l'agent a utilisé des outils.
- Taux d'erreur : exceptions ou réponses invalides.
- Qualitatif (notation humaine) : exactitude, complétude, cohérence.

9. Erreurs communes et comment les comprendre

- L'agent invente une réponse (hallucination) : il génère du contenu non vérifié.
- Outil indisponible : faut prévoir que l'appel à un outil puisse échouer et gérer ce cas.
- Ambiguïté dans le prompt : l'agent peut choisir une mauvaise stratégie si la consigne est floue.

10. Bonnes pratiques pour rédiger des tâches et prompts

- Être précis : dire exactement ce que vous attendez en sortie.
- Fournir des exemples de sortie si possible.
- Limiter la complexité d'une tâche pour diagnostiquer les erreurs pas à pas.
- Décrire le format de la réponse (par ex. JSON simple) pour faciliter l'évaluation automatique.

11. Glossaire (court)

- Agent : programme qui agit pour accomplir une tâche.
- Framework : boîte à outils logicielle pour construire des agents.
- RAG : méthode qui combine récupération de documents et génération.
- Orchestration : coordination des étapes et des appels d'outils.

12. Prochaines étapes recommandées

- Lire le protocole de recherche (document principal) pour comprendre la méthodologie.
- Choisir 5 tâches simples et les formuler clairement.

- Exécuter chaque tâche sur un framework à la fois, en enregistrant prompts, logs et temps

13. Ressources utiles (pour aller plus loin)

- Tutoriels d'introduction aux LLMs (recherchez "Introduction to Large Language Models")
- Documentation officielle : LangChain, AutoGen, LlamaIndex, CrewAI.

Protocole de recherche et d'évaluation comparative des frameworks d'agents IA

1. Contexte et justification

L'émergence des frameworks d'agents IA, notamment LangChain, CrewAI, AutoGen et LlamaIndex

Cette étude vise à proposer un protocole rigoureux pour comparer ces frameworks selon plusieurs critères

2. Objectif général

L'objectif principal de ce projet est de concevoir et d'appliquer un benchmark comparatif

3. Questions de recherche

Ce travail s'articule autour des questions de recherche suivantes :

1. Quel framework offre le meilleur taux de réussite global sur des tâches d'agents IA simples et complexes ?
2. Quel framework présente la latence la plus faible dans des scénarios simples et multi-agents ?
3. Quel framework est le plus robuste face aux erreurs d'outils, aux ambiguïtés ou aux changements de contexte ?
4. Quel framework est le plus adapté aux scénarios de collaboration multi-agents ?
5. Dans quelles conditions chaque framework devient-il le plus pertinent ?

4. Hypothèses de recherche

Les hypothèses suivantes guideront l'analyse :

- H1 : AutoGen sera plus performant dans les scénarios multi-agents grâce à son orienté agent.
- H2 : LangChain offrira une grande flexibilité et de bonnes performances sur les pipelines.
- H3 : CrewAI sera particulièrement efficace pour des workflows structurés par rôles et tâches.
- H4 : LlamaIndex montrera de meilleurs résultats dans les tâches reposant sur la récupération d'informations.

5. Périmètre de l'étude

L'étude se limite aux frameworks suivants :

- LangChain
- CrewAI

- AutoGen
- LlamaIndex

Le benchmark portera sur des tâches d'agents IA réalistes, mais volontairement circonscrits.

6. Conception du benchmark

6.1 Catégories de tâches

Le benchmark sera organisé en cinq catégories de tâches :

1. Tâches de recherche d'information
 - Identifier une information précise dans une source donnée.
 - Extraire et reformuler des éléments pertinents.
2. Tâches de raisonnement séquentiel
 - Résoudre un problème nécessitant plusieurs étapes logiques.
 - Enchaîner correctement plusieurs appels ou décisions.
3. Tâches de récupération documentaire
 - Répondre à partir d'un corpus de documents.
 - Synthétiser plusieurs passages pour produire une réponse cohérente.
4. Tâches multi-agents
 - Répartir des rôles entre plusieurs agents spécialisés.
 - Produire une réponse collaborative ou coordonnée.
5. Tâches avec ambiguïtés ou erreurs contrôlées
 - Tester la capacité du framework à gérer des entrées incomplètes.
 - Mesurer la résilience face à des outils indisponibles ou à des sorties incorrectes.

6.2 Nombre de tâches

Pour assurer un bon équilibre entre profondeur d'analyse et faisabilité, il est recommandé de limiter le nombre de tâches à 10.

7. Variables expérimentales

Afin de garantir la validité de la comparaison, les paramètres suivants devront être identiques pour tous les agents :

- même modèle de langage sous-jacent
- même température de génération
- même limite de tokens
- mêmes outils disponibles
- même budget maximal d'appels aux outils
- même timeout par tâche

- même format d'entrée et de sortie
- même nombre d'essais par tâche

Le contrôle strict de ces variables est indispensable pour éviter qu'une différence obse

8. Métriques d'évaluation

L'évaluation reposera sur des indicateurs quantitatifs et qualitatifs.

8.1 Métriques quantitatives

- Taux de réussite : proportion de tâches entièrement réussies.
- Taux de complétion partielle : proportion de tâches seulement partiellement réalisées.
- Latence moyenne : temps total nécessaire pour terminer une tâche.
- Nombre d'appels aux outils : quantité d'interactions effectuées par l'agent.
- Taux d'erreur : fréquence des exceptions, blocages ou sorties invalides.
- Consommation de ressources : mémoire et CPU si la mesure est disponible.

8.2 Métriques qualitatives

- Complétude de la réponse
- Pertinence du résultat
- Cohérence du raisonnement
- Robustesse face à l'incertitude
- Lisibilité de la sortie finale

9. Grille de notation

Une grille simple de notation peut être utilisée pour l'évaluation manuelle de chaque ré

- 0 : échec total
- 1 : résultat insuffisant
- 2 : résultat acceptable mais incomplet
- 3 : résultat correct et satisfaisant

Cette notation peut être appliquée selon trois critères principaux :

- exactitude
- complétude
- cohérence

La moyenne des trois critères donnera une note globale par tâche.

10. Méthodologie expérimentale

10.1 Préparation

Chaque tâche devra être rédigée de manière claire, testable et identique pour tous les f

10.2 Exécution

Chaque tâche sera exécutée plusieurs fois, idéalement entre 3 et 5 répétitions par frame

10.3 Collecte des données

Pour chaque exécution, les éléments suivants seront enregistrés :

- le prompt initial
- les appels outils effectués
- le temps total d'exécution
- les erreurs éventuelles
- la réponse finale produite
- la note attribuée à la réponse

10.4 Analyse

Les résultats seront ensuite agrégés par framework et par type de tâche. Une analyse com

11. Plan d'analyse des résultats

L'analyse des résultats devra répondre aux points suivants :

- Quel framework obtient le meilleur taux de réussite global ?
- Quel framework est le plus rapide ?
- Quel framework utilise le moins d'appels aux outils ?
- Quel framework est le plus robuste aux erreurs ?
- Quel framework est le plus efficace dans les tâches multi-agents ?

Les résultats pourront être présentés sous forme de :

- tableaux comparatifs
- graphiques en barres
- courbes de latence
- tableaux d'erreurs fréquentes
- synthèse qualitative des comportements observés

12. Reproductibilité

La reproductibilité est un élément central du protocole. À cet effet, il est recommandé

- la version de chaque framework
- la version du modèle utilisé
- les paramètres de génération
- la liste complète des outils
- les jeux de tâches utilisés
- les scripts d'exécution
- les logs bruts des expériences

Toutes les configurations devront être archivées pour permettre une reproduction ultérieure

13. Limites attendues

Ce protocole présente certaines limites qu'il conviendra de reconnaître dans le mémoire

- les performances peuvent varier selon le modèle de langage utilisé
- les résultats dépendent de la qualité de définition des tâches
- certaines métriques qualitatives restent partiellement subjectives
- les coûts de calcul peuvent limiter le nombre d'expériences

Ces limites ne remettent pas en cause la validité de l'étude, mais doivent être explicitées

14. Structure recommandée du mémoire

1. Introduction générale
2. État de l'art sur les agents IA et les frameworks
3. Problématique et objectifs
4. Méthodologie expérimentale
5. Description du benchmark
6. Résultats expérimentaux
7. Analyse comparative
8. Discussion des limites
9. Conclusion et perspectives

15. Conclusion

Ce protocole propose une base méthodologique claire, reproductible et académique pour co